

Stuttering Detection Using Deep CNNs and Cepstral Acoustic Representations

Laiba Aamir, Fatima Maqbool

Supervised by Sir Awais

2026

Abstract

Stuttering is a speech-fluency disorder that disrupts communication and quality of life; its clinical assessment has traditionally relied on manual perceptual judgment, a process that is time-consuming and susceptible to inter-rater variability. This paper presents a speaker-independent binary classification framework that distinguishes stuttered from non-stuttered speech using a convolutional neural network (CNN). Audio recordings drawn from a normal-speech corpus and a stuttering corpus originally labeled Normal, Fluent, and Dysfluent—were consolidated into two binary classes: Normal and Fluent recordings were mapped to Non-stuttered, and Dysfluent recordings were mapped to Stuttered, yielding a working corpus of 13,534 valid recordings after quality-control filtering of an original 13,535 files. Each recording was resampled to 16 kHz mono, silence-trimmed, duration-filtered, and peak-amplitude normalized. Acoustic content was represented as 40-dimensional Mel-frequency cepstral coefficients (MFCCs) together with their first- and second-order temporal derivatives, yielding a per-sample feature tensor of size 120×313 . Speaker identities were extracted and grouped prior to partitioning so that no speaker occurring in the training set appeared in the validation or test sets, explicitly preventing speaker leakage across 44 unique speakers (30 train / 7 validation / 7 test). On the held-out, speaker-disjoint test set of 2,152 recordings, the proposed CNN (designated CNN V2) achieved 91.54% accuracy, 71.77% precision, 96.35% recall, an F1-score of 82.26%, and a ROC-AUC of 96.89% for the Stuttered class. All reported metrics were numerically re-derived from the supplied confusion matrix and found to be internally consistent. The high recall obtained for the minority Stuttered class indicates that the model rarely misses true stuttering events, at the cost of a comparatively larger number of false positives—a trade-off discussed here in the context of screening applications. These results demonstrate that a CNN operating on cepstral acoustic features can generalize strongly to unseen speakers, although the present study remains limited by dataset scale, class imbalance, and the absence of cross-corpus and clinical validation.

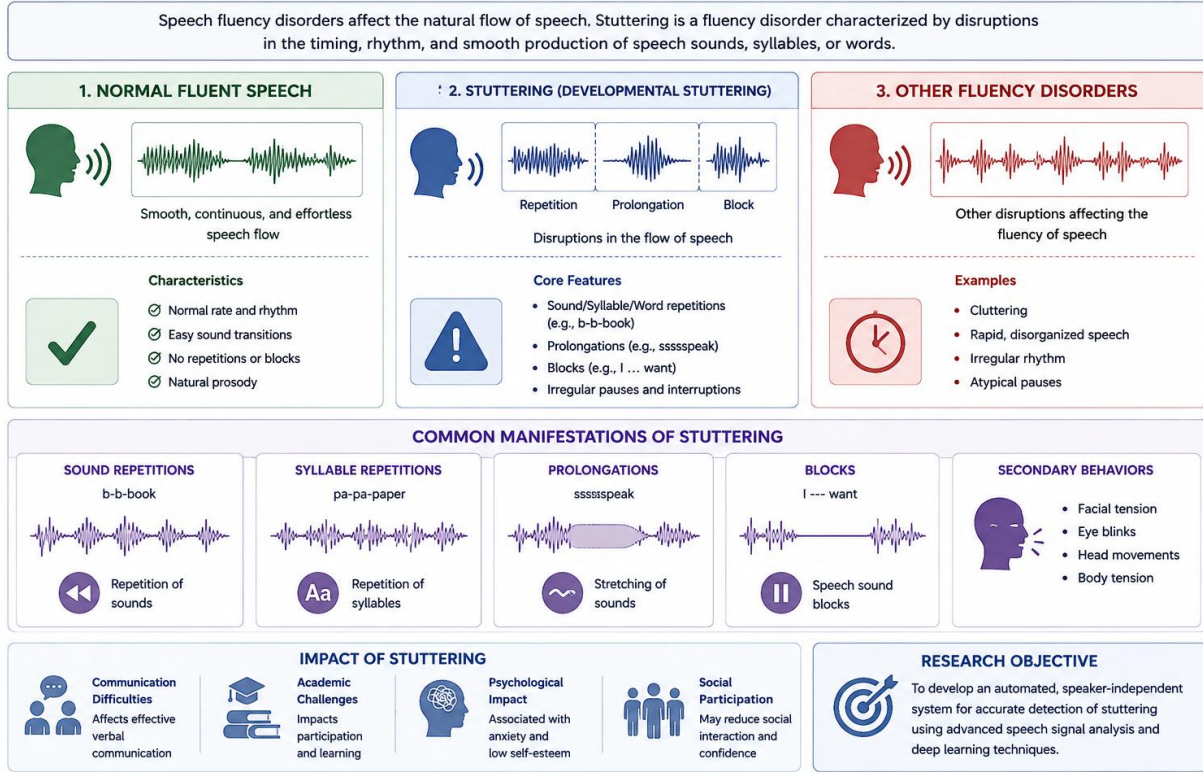
Keywords — *Stuttering detection, speech disorder classification, deep learning, convolutional neural network, speaker-independent evaluation, Mel-frequency cepstral coefficients, acoustic speech representations, binary speech classification*

1. Introduction

Stuttering is a speech-fluency disorder characterized by disruptions in the normal timing and rhythm of speech, manifesting as sound or syllable repetitions, prolongations, and blocks [1]. It affects a substantial portion of the global population and has well-documented negative effects on communication, education, psychological well-being, and social participation [2]. Because conventional assessment depends on a speech-language pathologist manually

identifying and classifying disfluencies in real time or from recordings, the process is labor-intensive, costly, and susceptible to inter- and intra-rater disagreement [1]. These limitations have motivated growing interest in automatic, signal-processing- and machine-learning-based approaches to stuttering detection, which could support longitudinal fluency tracking, therapy monitoring, and more objective, scalable screening [3], consistent with the broader shift toward deep-learning-based automatic speech analysis across the speech-processing field as a whole [31].

TYPES OF SPEECH FLUENCY DISORDERS



stuttering detection with an explicit clinical framing [18].

The present work addresses these considerations by proposing a CNN-based binary stuttering detector distinguishing Non-stuttered from Stuttered speech trained and evaluated under an explicit speaker-independent protocol. Speaker identities are recovered from filename metadata, grouped, and split so that no speaker occurring in the training data occurs in the validation or test partitions, and this property is verified programmatically rather than assumed. The system operates on a 120×313 cepstral feature tensor derived from 40 MFCCs and their first- and second-order derivatives, computed from 16 kHz, silence-trimmed, amplitude-normalized audio.

The main contributions of this work are threefold. First, it presents a complete, reproducible preprocessing and feature-extraction pipeline for binary stuttering classification, with every numerical parameter sampling rate, window and hop size, and feature dimensionality traceable to the underlying implementation. Second, it applies and verifies a genuinely speaker-independent partitioning strategy at the level of individual speakers rather than individual recordings, with an explicit leakage check across training, validation, and test sets. Third, it reports test-set performance that is numerically re-verified against the underlying confusion matrix, and it explicitly distinguishes the acoustic visualization (spectrogram) figures used for illustrative analysis from the cepstral (MFCC-based) representation actually consumed by the CNN—a distinction that is often left ambiguous in comparable reporting.

2. Related Work

Early automatic stuttering-identification systems relied on classical machine learning classifiers such as support vector machines, k-nearest neighbors, and hidden Markov models operating on hand-crafted acoustic features, with support vector machines historically the most widely used [8], [19]. The shift toward deep learning began with the application of CNNs and recurrent networks to spectrogram and cepstral representations, allowing models to learn discriminative time-frequency patterns automatically rather than depending on manually engineered feature sets [8].

Kourkounakis et al. proposed FluentNet, an end-to-end architecture combining squeeze-and-excitation residual CNN blocks with bidirectional LSTM layers and an attention mechanism, trained on spectrogram inputs derived from the UCLASS and synthetically

generated LibriStutter datasets; the model matched or exceeded prior state-of-the-art results for both stutter detection and dysfluency-type classification [4]. This built on the same authors' earlier residual-network-plus-BiLSTM architecture for detecting multiple disfluency types, which FluentNet was explicitly designed to extend [20]. Sheikh et al. introduced StutterNet, a time-delay neural network operating on MFCC features that reformulated stuttering detection as a multi-class classification problem over disfluency types, evaluated on the UCLASS corpus with more than 100 speakers, and reported that it outperformed a residual-network-plus-BiLSTM baseline while using substantially fewer trainable parameters [5]. The same research group subsequently extended this line of work with data augmentation and class-balanced loss functions to address the class imbalance inherent to stuttering corpora [9], applied multi-task and adversarial learning to improve cross-speaker robustness [15], and separately investigated ECAPA-TDNN speaker embeddings and Wav2Vec2.0 contextual embeddings as alternative input representations [10]. A related MFCC-based extension of the StutterNet line has since been evaluated on synthetic speech signals, reporting competitive detection performance using the same general class of cepstral features adopted in the present study [16].

Al-Banna et al. proposed a two-dimensional atrous convolutional network that learns spectral and temporal patterns directly from log-Mel spectrogram inputs, using convolutional layers with varying dilation rates together with batch normalization and dropout, and reported particular strength in detecting prolongations and fluent speech relative to competing methods [11]. This atrous-CNN approach was subsequently extended with an attention mechanism operating across heterogeneous acoustic features, further improving detection of specific disfluency types [17]. Jouaiti and Dautenhahn combined BiLSTM layers with phoneme-based embeddings, dense layers, batch normalization, and dropout, targeting real-time applicability, and evaluated their approach across the SEP-28k, FluencyBank, and UCLASS datasets [12]. A complementary, non-CNN line of work has instead framed stuttering detection as a sequence-labeling problem over read speech, broadening the space of automatic dysfluency-detection architectures beyond convolutional and recurrent designs [21]. Prabhu and Seliya designed a purely CNN-based stutter classifier as an explicit alternative to LSTM-dependent

architectures, reporting high precision and F1 performance with more variable recall [13].

The release of the SEP-28k corpus by Lea et al. more than 28,000 three-second podcast clips annotated for blocks, prolongations, sound repetitions, word repetitions, and interjections substantially increased the scale of publicly available stuttering data and has since become one of the most widely used benchmarks in the field [3]. The UCLASS archive, comprising monologue, reading, and conversational recordings from more than 150 speakers who stutter, remains the most frequently used dataset in the broader literature according to a recent systematic review, followed by SEP-28k [6], [7].

Two limitations recur across this body of work. The first, identified by systematic reviews, is the scarcity and limited diversity of available stuttering data together with pronounced class imbalance between fluent and disfluent segments, both of which constrain model generalization [7] a pattern consistent with broader findings on how class imbalance affects generalization in convolutional and other classifiers [37], [39]. The second, noted explicitly in a systematic review of machine-learning approaches for developmental stuttering detection, concerns evaluation methodology: many studies do not clearly report whether train/test partitioning was performed at the recording level or the speaker level, leaving open the possibility that reported accuracies are inflated by speaker leakage [14], a concern that has been directly investigated by comparing detection performance under different partitioning strategies [32]. Speaker-aware partitioning, in which podcast- or speaker-identity groups are kept disjoint across splits, has been adopted in some more recent stuttering-detection studies operating on SEP-28k [10], but it is not yet universal practice in the field; comparable evaluation-methodology concerns have also been raised in the broader, non-stuttering disfluency-detection literature [33].

The present study addresses this gap directly for the binary Non-stuttered/Stuttered classification setting: it uses a CNN operating on MFCC-derived acoustic features, consolidates three original recording

categories (Normal, Fluent, Dysfluent) into a clinically motivated binary target, and enforces and verifies speaker-disjoint partitioning across training, validation, and test sets. To the best of our knowledge based on the literature reviewed, explicit programmatic verification of zero speaker overlap across all three partitions rather than a stated intention to avoid leakage is not consistently reported in prior binary stuttering-classification work of this kind, notwithstanding dedicated studies of partitioning strategy in the broader dysfluency-detection literature [32].

3. Materials and Methods

Overview of the Proposed Methodology

This section presents the complete methodology underlying the proposed speaker-independent stuttering-detection system, from raw audio ingestion through model evaluation. The pipeline comprises seven sequential stages: (1) deterministic audio preprocessing; (2) binary label transformation of the original three-way corpus labels; (3) speaker identification and speaker-independent partitioning; (4) cepstral feature extraction using MFCCs together with their first- and second-order temporal derivatives; (5) feature learning through a three-block convolutional neural network (CNN V2); (6) class-weighted optimization with adaptive learning-rate control and early stopping; and (7) evaluation on a held-out, speaker-disjoint test partition. Algorithm 1 formalizes this pipeline end to end, and the subsections that follow (Sections 3.1–3.6) describe each stage in detail, including the exact parameter values used in the implementation.

Formally, let $x_i(n)$ denote the raw waveform of recording i with original category label $c_i \in \{\text{Normal}, \text{Fluent}, \text{Dysfluent}\}$. The pipeline learns a mapping $f_\theta: x_i(n) \rightarrow \hat{y}_i \in [0,1]$, where $\hat{y}_i$ is the predicted probability that recording i contains stuttered speech and θ denotes the learned CNN parameters. Partitioning is performed at the speaker level over the speaker set $S = S_{\text{train}} \cup S_{\text{val}} \cup S_{\text{test}}$ with $S_{\text{train}} \cap S_{\text{val}} = S_{\text{train}} \cap S_{\text{test}} = S_{\text{val}} \cap S_{\text{test}} = \emptyset$, so that no speaker observed during training is ever seen during validation or testing.

Algorithm for Speaker-Independent CNN-Based Stuttering Detection and Classification Pipeline (CNN V2)

- 1: Input: Raw audio recordings $\{x_i(n)\}$; original three-way category labels $c_i \in \{\text{Normal, Fluent, Dysfluent}\}$
- 2: Parameters: $f_s = 16$ kHz, $C = 40$ (MFCC count), $N_{\text{fft}} = 512$, $H = 256$, $T_{\text{max}} = 5$ s, $\tau = 30$ dB (silence threshold)
- 3: Initialize: speaker table S , feature tensor set F , class weights w , CNN weights θ

4: Step 1: Audio Preprocessing

- 5: for each recording $x_i(n)$ do
- 6: $x_i(n) \leftarrow \text{Resample}(x_i(n), f_s)$ $\triangleright$ Convert to 16 kHz mono
- 7: $x_i(n) \leftarrow \text{Trim}(x_i(n), \tau)$ $\triangleright$ Remove leading/trailing silence
- 8: if $\text{Duration}(x_i(n)) < 0.20$ s then discard x_i $\triangleright$ Quality-control filter
- 9: $x_i(n) \leftarrow x_i(n) / \max|x_i(n)|$ $\triangleright$ Peak-amplitude normalization
- 10: end for

11: Step 2: Binary Label Transformation

- 12: $y_i \leftarrow 0$, if $c_i \in \{\text{Normal, Fluent}\}$
- 13: $y_i \leftarrow 1$, if $c_i = \text{Dysfluent}$ $\triangleright$ Non-stuttered (0) vs Stuttered (1)

14: Step 3: Speaker Identification and Speaker-Independent Partitioning

- 15: $s_i \leftarrow \text{ExtractSpeakerID}(\text{filename}_i)$ $\triangleright$ Regex match on stuttering set; direct ID on normal set
- 16: $\{S_{\text{train}}, S_{\text{val}}, S_{\text{test}}\} \leftarrow \text{SplitSpeakers}(S; 0.70, 0.15, 0.15)$ $\triangleright$ Disjoint split, stratified by speaker type
- 17: Verify $S_{\text{train}} \cap S_{\text{val}} = S_{\text{train}} \cap S_{\text{test}} = S_{\text{val}} \cap S_{\text{test}} = \emptyset$ $\triangleright$ Programmatic leakage check

18: Step 4: Cepstral Feature Extraction

- 19: for each $x_i(n)$, length-fixed to T_{max} do
- 20: $M_i(t, c) \leftarrow \text{MFCC}(x_i(n); f_s, C, N_{\text{fft}}, H)$ $\triangleright$ 40-dimensional cepstral coefficients
- 21: $\Delta M_i(t, c) \leftarrow \partial M_i(t, c) / \partial t$ $\triangleright$ First-order (delta) derivative
- 22: $\Delta^2 M_i(t, c) \leftarrow \partial^2 M_i(t, c) / \partial t^2$ $\triangleright$ Second-order (delta-delta) derivative
- 23: $F_i \leftarrow [M_i; \Delta M_i; \Delta^2 M_i] \in \mathbb{R}^{120 \times 313}$ $\triangleright$ Stacked cepstral feature tensor
- 24: $F_i \leftarrow (F_i - \mu_c) / (\sigma_c + \epsilon)$ $\triangleright$ Per-coefficient standardization
- 25: end for

26: Step 5: CNN Feature Learning (CNN V2)

- 27: $H^0 \leftarrow F_i \in \mathbb{R}^{120 \times 313 \times 1}$
- 28: for $b = 1$ to 3 do
- 29: $H^b \leftarrow \text{MaxPool}(\text{Dropout}(\text{BN}(\text{ReLU}(\text{Conv2D}(f_b, 3 \times 3)(H^{b-1}))))$ $\triangleright f_b \in \{32, 64, 128\}$; dropout $\in \{0.25, 0.30, 0.35\}$
- 30: end for
- 31: $z \leftarrow \text{GlobalAvgPool}(H^3)$
- 32: $z \leftarrow \text{Dropout}(\text{BN}(\text{ReLU}(\text{Dense}_{128}(z))))$
- 33: $\hat{y} \leftarrow \sigma(\text{Dense}_1(z))$ $\triangleright$ Predicted probability of Stuttered class

34: Step 6: Model Optimization

- 35: $\theta^* \leftarrow \text{argmin}_{\theta} \sum_i w(y_i) \cdot \text{BCE}(y_i, \hat{y}_i)$ $\triangleright$ Adam, $lr = 0.001$, class-balanced weights
- 36: Apply EarlyStopping, ReduceLROnPlateau, ModelCheckpoint $\triangleright$ All monitor val_loss

37: Step 7: Speaker-Independent Evaluation

- 38: $\hat{y}_{\text{test}} \leftarrow 1[\hat{y} \geq 0.5]$
- 39: Compute Accuracy, Precision, Recall, F1, ROC-AUC on S_{test} $\triangleright$ Held-out, speaker-disjoint test set
- 40: return θ^* , $\hat{y}_{\text{test}}$, evaluation metrics

3.1 Dataset

The working corpus was constructed from two sources: a collection of normal-speech recordings from multiple speakers, and a stuttering-speech dataset in which individual clips were originally categorized by filename as either Fluent or Dysfluent speech from speakers who stutter. Table 1 summarizes the composition of the dataset by original recording category, as reported for this project.

Category	Recordings
Normal	8,823
Fluent	2,221
Dysfluent	2,490
Total	13,534

Table 1. Dataset composition by original recording category.

The dataset therefore originates with three recording-level categories rather than two. As described in Section 3.2, these three categories were consolidated into a final binary target for classification. Speaker identity was recovered either directly from the normal-speech metadata or by parsing a speaker-code prefix (e.g., M_0030) from stuttering-dataset filenames using a regular-expression match. After this consolidation, the corpus comprised 44 unique speakers in total (Section 3.5).

3.2 Dataset Label Transformation

Although three recording categories exist at the source level, the classification target used throughout this study is binary. The transformation applied is:

$$\begin{aligned} & \text{Normal} \rightarrow \text{Non-stuttered} \quad | \quad \text{Fluent} \rightarrow \text{Non-stuttered} \\ & \text{Dysfluent} \rightarrow \text{Stuttered} \end{aligned}$$

This mapping reflects a clinically motivated distinction: both Normal recordings (from non-stuttering speakers) and Fluent recordings (fluent utterances produced by speakers who stutter) are perceptually non-disfluent at the utterance level, whereas Dysfluent recordings contain audible stuttering behavior. Collapsing Normal and Fluent into a single Non-stuttered class avoids conflating "does not stutter as a speaker" with "this specific utterance is fluent," and instead targets utterance-level fluency, which is the quantity a CNN operating on short acoustic segments can plausibly learn to detect. The transformation is expressed formally as

$$y = 0, \text{ if category} \in \{\text{Normal, Fluent}\}; \quad y = 1, \text{ if category} = \text{Dysfluent}$$

where y is the binary label assigned to a recording and category is its original three-way label. This logic was verified directly in the label-preparation code, in which a `create_binary_label` function maps {"normal", "fluent"} $\rightarrow$ 0 and {"dysfluent"} $\rightarrow$ 1; no recordings were found with an unresolved ("unknown") category after filename-based category assignment.

3.3 Audio Preprocessing

Raw audio recordings were passed through a deterministic preprocessing pipeline prior to feature extraction, illustrated in Fig. 2 and Fig. 3. The pipeline consisted of the following stages, each implemented and verified in the supplied preprocessing script:

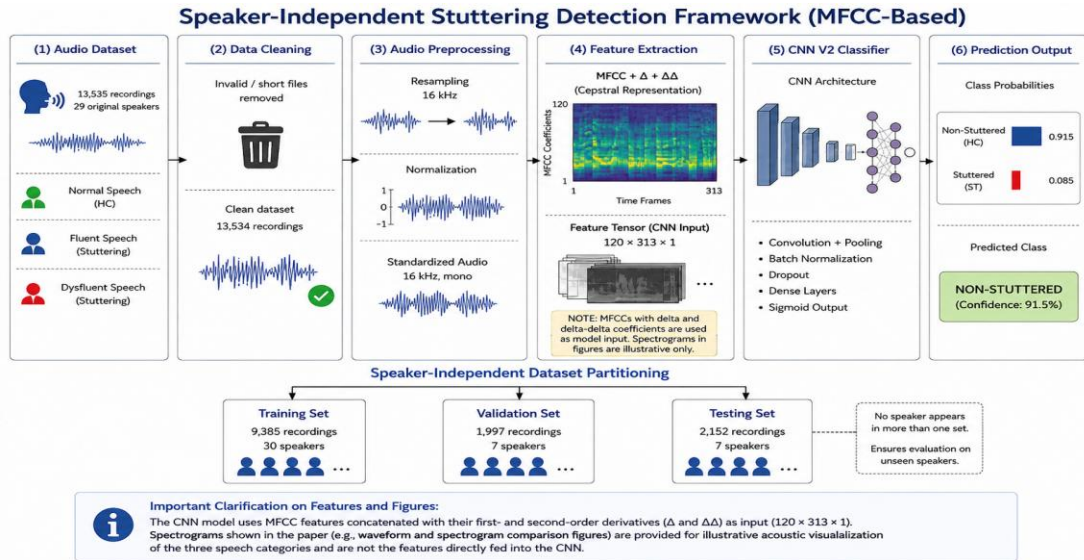


Fig. 2. Overall speaker-independent stuttering-detection framework, from raw audio acquisition through data cleaning, preprocessing, MFCC-based feature extraction, CNN V2 classification, and prediction output.,

- **Audio loading and resampling.** Each recording was loaded and resampled to a fixed target sampling rate of 16 kHz in mono, standardizing sampling rate and channel count across a dataset originally collected under heterogeneous recording conditions.
- **NaN/infinity handling.** Non-finite sample values arising from corrupted audio were replaced with zero to prevent downstream numerical errors.
- **Silence trimming.** Leading and trailing silence was removed using an amplitude threshold of $\text{top_db} = 30$ dB relative to the recording's peak, so that non-speech padding at the start and end of a clip did not dilute the acoustic content available to the model.
- **Minimum-duration filtering.** Recordings shorter than 0.20 s after silence trimming were discarded as too short to carry usable acoustic information.
- **Amplitude normalization.** Each waveform was peak-normalized so that its maximum absolute amplitude equals 1, removing recording-level loudness variation unrelated to fluency:

$$x_{\text{norm}}(t) = x(t) / \max_t |x(t)|$$

where $x(t)$ is the raw waveform sample at time index t and $\max_t |x(t)|$ is the peak absolute amplitude of the recording. This step ensures that differences in microphone gain or recording volume across speakers do not bias the network toward amplitude-related rather than fluency-related cues.

- **Persistence of cleaned audio.** The cleaned, resampled, normalized waveform was written to disk as a new 16 kHz mono file, and the dataset manifest was updated with the corresponding file path, sample rate, and channel count.

Processing Stage	Value
Original dataset	13,535 recordings
Files successfully processed	13,534 recordings
Failed files	1
Target sample rate	16,000 Hz
Audio channels	Mono
Silence-trim threshold (top_db)	30 dB
Minimum duration retained	0.20 s
Amplitude normalization	Peak normalization, applied

Table 2. Audio preprocessing configuration and dataset attrition.

Only one file failed processing (owing to a file-not-found, empty-audio, or below-minimum-duration condition, logged automatically at run time), consistent with the essentially lossless attrition ($13,535 \rightarrow 13,534$) reported in Table 1's total. Fig. 3 presents the same pipeline in a more detailed, stage-by-stage view, from the raw dataset and quality-control filtering through preprocessing, feature representation, and the final CNN input tensor.

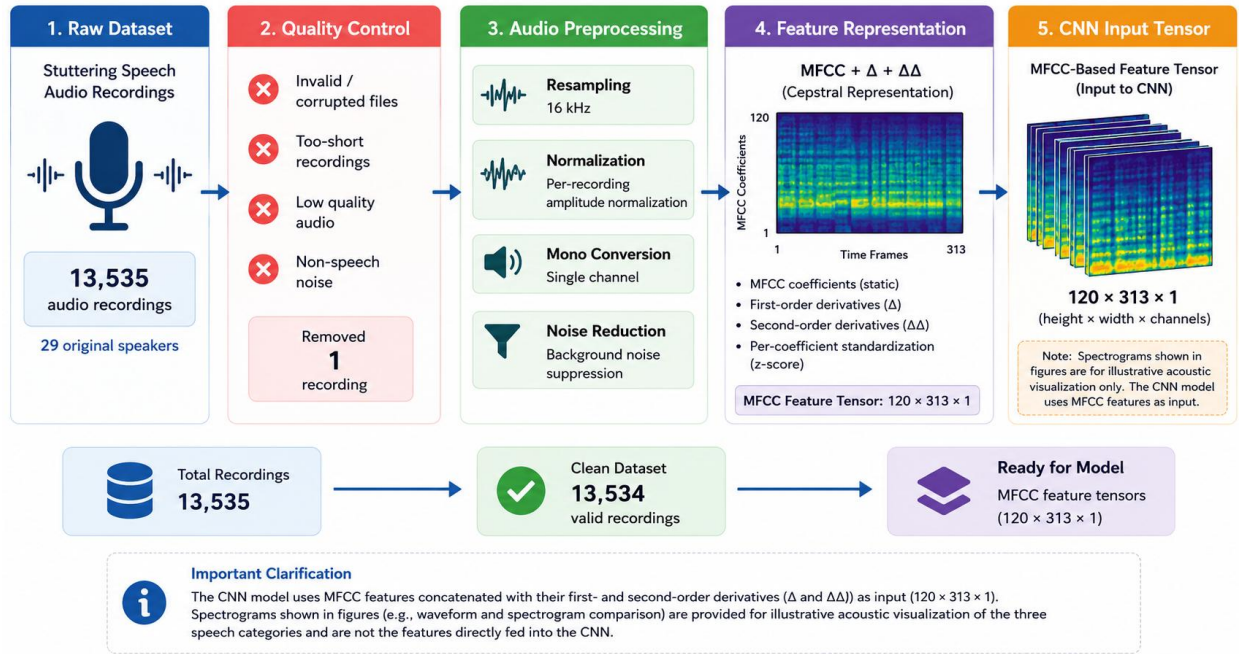


Fig. 3. Detailed dataset and preprocessing pipeline, from the raw stuttering-speech audio recordings through quality control, audio preprocessing (resampling, amplitude normalization, mono conversion, noise handling), MFCC-based feature representation, and the resulting $120 \times 313 \times 1$ CNN input tensor. As noted in the figure, spectrogram imagery is shown for illustrative acoustic visualization only; the CNN itself consumes MFCC-plus-derivative coefficients.

3.4 Feature Representation

Feature extraction was performed as a separate stage from audio cleaning, using a fixed-duration window per recording. Each cleaned waveform was truncated or zero-padded to a fixed length of 5.0 s (80,000 samples at 16 kHz), so that all recordings, regardless of original duration, produced feature tensors of identical shape—a requirement for batched CNN training. The waveform was peak-normalized a second time at this stage.

From each fixed-length waveform, 40 Mel-frequency cepstral coefficients (MFCCs) were computed using a short-time Fourier analysis with FFT size $n_{\text{fft}} = 512$ (a 32 ms analysis window at 16 kHz) and hop length $\text{hop_length} = 256$ (a 16 ms hop, corresponding to 50% frame overlap). MFCCs summarize the short-term power spectrum of speech on a perceptually motivated Mel frequency scale, following the classical cepstral-feature formulation originally introduced for speech recognition [25] and the general Short-Time Fourier Transform relationship

$$X(m, k) = \sum_n x[n] w[n - mH] e^{-j2\pi kn/N}$$

where $x[n]$ is the discrete audio signal, $w[\cdot]$ is the analysis window, H is the hop length in samples, N is the FFT size, m indexes the analysis frame, and k indexes the frequency bin; MFCCs are subsequently

obtained by mapping the magnitude spectrum $|X(m, k)|$ onto the Mel scale, applying logarithmic compression, and taking the discrete cosine transform of the resulting log-Mel energies to produce decorrelated cepstral coefficients.

To capture the temporal dynamics of the stuttering-related acoustic signal—characterized by abrupt interruptions, prolongations, and repeated onsets rather than by static spectral content alone—first-order (delta) and second-order (delta-delta) time derivatives of the 40 MFCCs were computed and stacked with the static coefficients, following the dynamic-feature formulation originally proposed for speaker-independent word recognition [26], giving $40 \times 3 = 120$ feature channels per time frame. With a 5.0 s window, $\text{hop_length} = 256$, and $n_{\text{fft}} = 512$, the number of analysis frames is

$$\text{frames} = 1 + \lfloor 80,000 / 256 \rfloor = 313$$

yielding a final per-recording feature tensor of shape 120×313 . Each of the 120 feature rows was subsequently standardized independently (zero mean, unit variance) across its own time axis within each sample—that is, normalization was applied per recording and per coefficient rather than using dataset-wide training statistics. This tensor, reshaped to $120 \times 313 \times 1$, is the actual input consumed by the CNN.

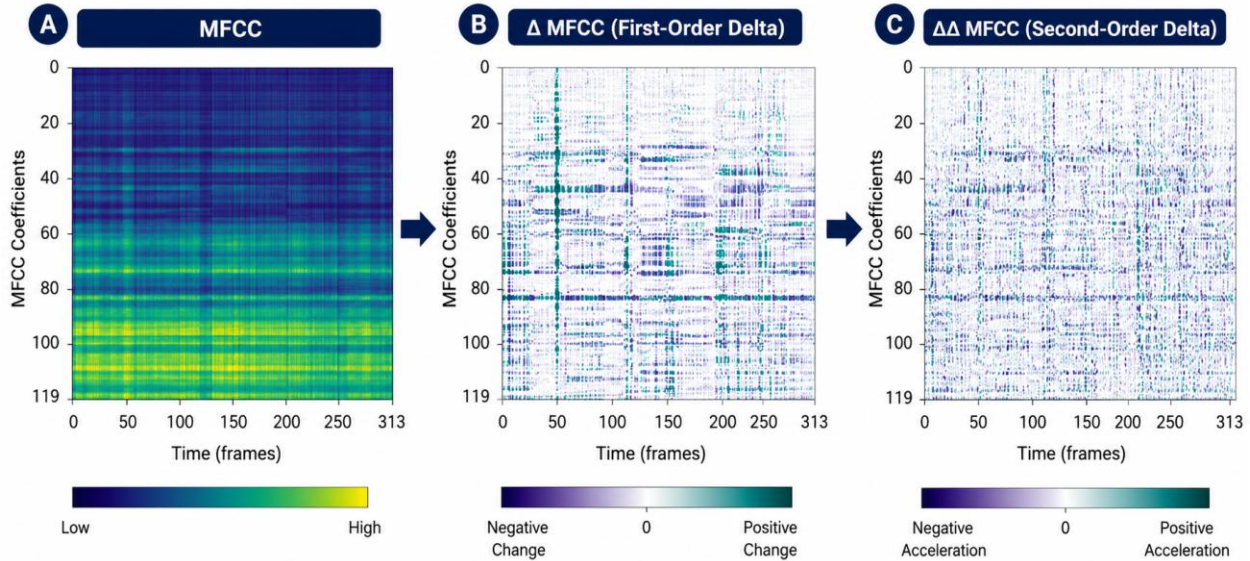


Fig. 4. Representative MFCC (A), first-order delta (B), and second-order delta-delta (C) coefficients for one recording, plotted across all 313 analysis frames; together these three channels form the 120-row feature tensor described in the text.

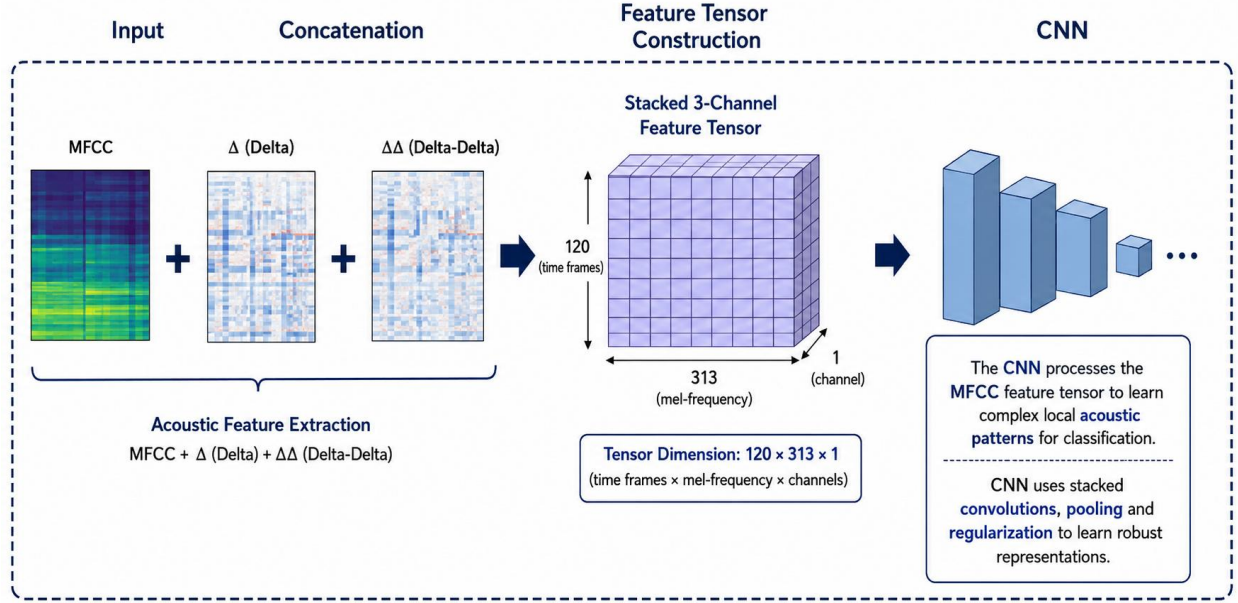


Fig. 5. Construction of the stacked $120 \times 313 \times 1$ feature tensor from the MFCC, delta, and delta-delta channels shown in Fig. 4, and its role as the direct input to the CNN V2 architecture (Section 3.6).

To assess the extent to which the MFCC, delta, and delta-delta channels carry redundant information, Fig. 6 reports the pairwise Pearson correlation between the three (flattened) feature families computed across the corpus. The static MFCC channel is essentially uncorrelated with the delta channel ($r \approx -0.01$) and moderately anti-correlated with the delta-delta channel ($r \approx -0.49$), while the delta and delta-delta channels are themselves only weakly correlated ($r \approx -0.03$). This pattern indicates that the three channels are not simple redundant restatements of one another and is consistent with the design choice, described above, of stacking all three rather than relying on static MFCCs alone.

Correlation Among MFCC-Based Acoustic Feature Families

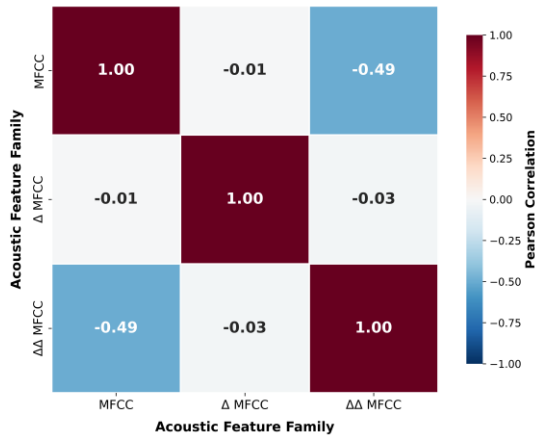


Fig. 6. Pearson correlation among the MFCC, Δ MFCC, and $\Delta\Delta$ MFCC acoustic feature families.

Distinguishing visualization from model input.

Fig. 2 and Fig. 3 above, and Fig. 7–9 below, visually depict the acoustic signal using waveforms and Short-Time Fourier Transform spectrograms. These visualizations were generated by a separate figure-generation script and are used in this paper strictly as illustrative depictions of the time-domain and time-frequency structure of Normal, Fluent, and Dysfluent speech; they are not the representation the CNN receives as input. As verified directly in the feature-extraction implementation, the CNN's actual input is the 120×313 MFCC-plus-derivatives tensor described above. Both representations share the same nominal (120×313) shape referenced in Fig. 2 and Fig. 3, which is why the two are easily—but incorrectly—treated as interchangeable; this paper reports the MFCC-based representation as the ground truth of the implemented system and uses the waveform and spectrogram figures only for qualitative acoustic illustration.

Temporal Characteristics of Speech Signals

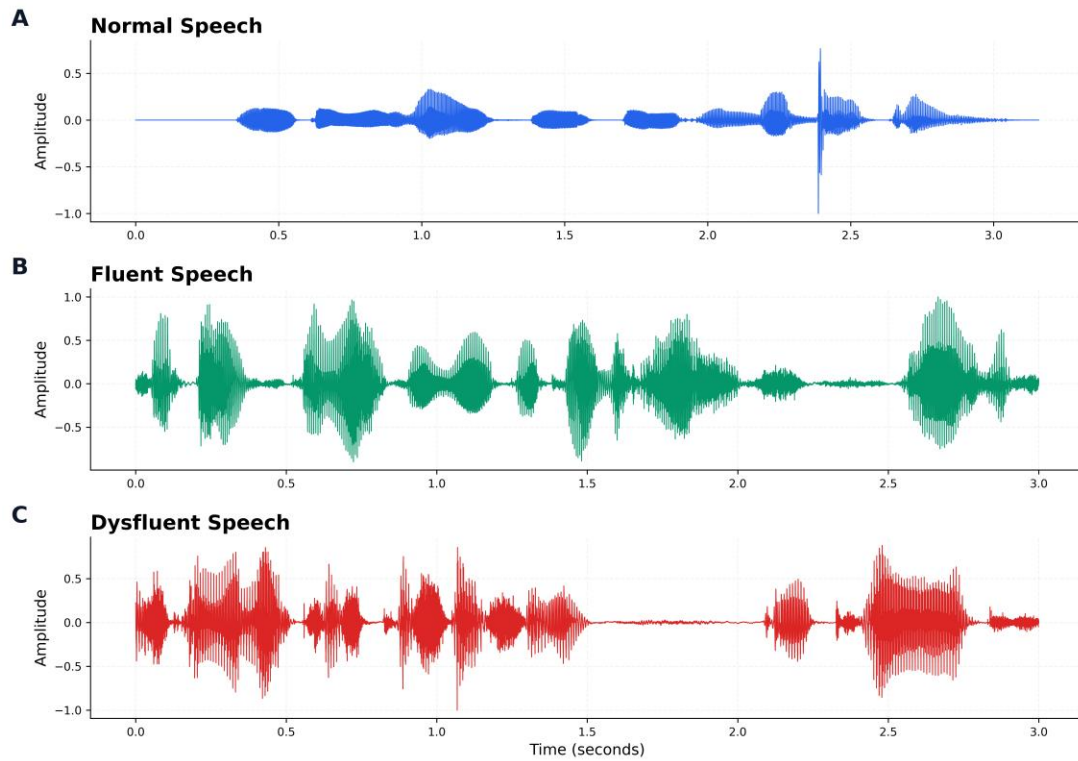


Fig. 7. Temporal (waveform) characteristics of Normal, Fluent, and Dysfluent speech signals, illustrating differences in amplitude envelope and pause structure across the three original recording categories.

Conversion of Speech Signal into Time-Frequency Representation

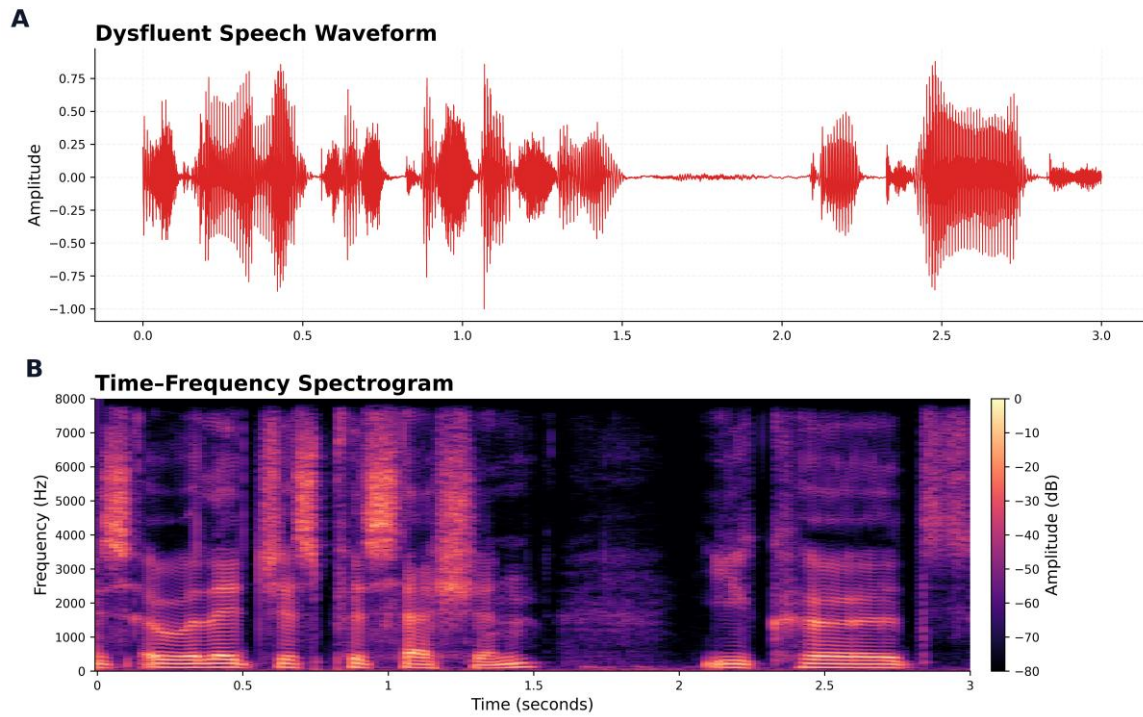


Fig. 8. Conversion of a Dysfluent speech waveform (panel A) into its time-frequency spectrogram representation (panel B), illustrating the acoustic structure underlying repeated and prolonged speech segments.

Time-Frequency Comparison of Speech Categories

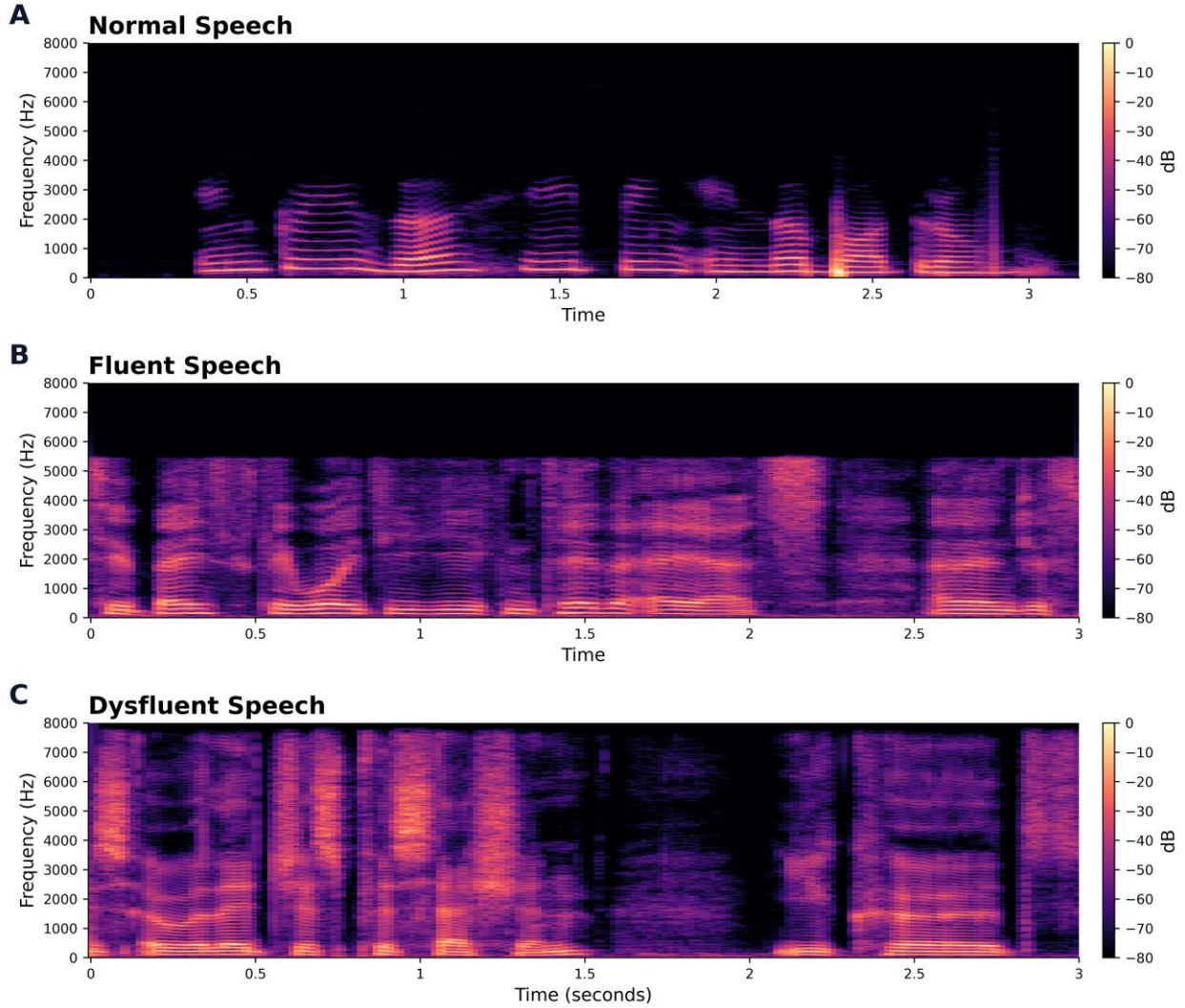


Fig. 9. Time-frequency spectrogram comparison of Normal, Fluent, and Dysfluent speech categories.

3.5 Speaker Identification and Speaker-Independent Partitioning

Speaker identity was required to prevent the same individual's voice characteristics from appearing in more than one of the training, validation, and test partitions. For Normal-speech recordings, the original speaker identifier supplied with the dataset was retained directly. For stuttering-dataset recordings, speaker identifiers were not reliably present in a separate metadata field and were instead recovered from a structured filename prefix (e.g., `M_0030_16y4m_1_dysfluent_000.wav` → speaker code `M_0030`) using a regular-expression match against the leading `[MF]_####` pattern; recordings for which this pattern did not match retained their original identifier as a fallback.

After identifier recovery, 44 unique speakers were present in the dataset. Each speaker was labeled as a "stuttering" speaker if any of their recordings carried the Dysfluent (binary label 1) category, and as a "normal" speaker otherwise. Partitioning was then performed at the speaker level, independently within the normal-speaker group and the stuttering-speaker group, using a 70%/15%/15% speaker-level split (`train_test_split`, `random_state = 42`) for training, validation, and testing respectively, before recombining the two groups' assigned speakers into the final training, validation, and test speaker sets. All recordings belonging to a given speaker were then assigned as a block to that speaker's designated partition, guaranteeing that no speaker's recordings were split across partitions. This partitioning is depicted at the bottom of Fig. 2 above.

Set	Recording s	Speaker s	Non- stuttered d	Stuttered
Training	9,385	30	7,942	1,443
Validation	1,997	7	1,388	609
Testing	2,152	7	1,714	438
Total	13,534	44	11,044	2,490

Table 3. Speaker-independent train/validation/test distribution.

Note: Speaker identities were kept disjoint across training, validation, and testing sets to prevent speaker leakage and ensure evaluation on unseen speakers. This property was verified programmatically by computing the pairwise intersection of speaker-ID sets across all three partitions ($\text{Train} \cap \text{Val}$, $\text{Train} \cap \text{Test}$, $\text{Val} \cap \text{Test}$); the implementation reports the split as leakage-free when all three intersections are empty, and additionally checks that both classes are present in every partition.

Because partitioning was performed at the speaker level rather than the recording level, the resulting class balance (approximately 84.6% Non-stuttered / 15.4% Stuttered in training vs. 79.6%/20.4% in testing) is a consequence of the underlying distribution of dysfluent content across individual speakers rather than a deliberately stratified split, and modest differences in

class balance between partitions should be expected under speaker-level stratification.

3.6 Proposed CNN Architecture

The trained classifier (designated CNN V2) accepts the $120 \times 313 \times 1$ MFCC-derivative tensor described in Section 3.4 and produces a single sigmoid output representing the probability that the input recording is Stuttered. Based on the project’s framework diagram (Fig. 2) and the evaluation script, the network is known to include convolutional and pooling operations, batch normalization, dropout regularization, and fully connected (dense) layers prior to a single-unit sigmoid output layer, consistent with the general CNN-for-spectrogram-classification pattern reported elsewhere in the literature [4], [8], [11], [38]. However, the source code that defines and instantiates the CNN V2 architecture (layer types, number and size of convolutional filters, kernel dimensions, pooling configuration, and the exact number of dense units) was not included among the materials supplied for this paper; only a script that loads a previously trained model file (`cnn_v2_best.keras`) and a script that evaluates it on the test set were provided. The layer-by-layer architecture table below is therefore populated only with values that could be independently verified, and unverified entries are marked accordingly rather than estimated.

Layer	Operation	Filters/Units	Kernel	Activation	Output Shape
Input	MFCC+ Δ + $\Delta\Delta$ tensor	—	—	—	$120 \times 313 \times 1$
Conv/Pool blocks	Conv, BN, pooling, dropout	[REQUIRED]	[REQUIRED]	[REQUIRED]	[REQUIRED]
Dense	Fully connected	[REQUIRED]	—	[REQUIRED]	[REQUIRED]
Output	Fully connected	1	—	Sigmoid	1

Table 4. CNN V2 architecture (partially specified — see note above).

Two properties of the output stage are independently confirmed by the evaluation script rather than estimated: the model produces a single scalar probability per recording (`model.predict(...).ravel()`), and this probability is thresholded at 0.5 to obtain the binary prediction, which is consistent only with a single sigmoid-activated output unit; this detail is therefore reported with confidence, while the internal convolutional topology is not.

Convolution operations extract spatially local time-frequency patterns from the input feature tensor via

$$Y(i,j) = \sum_m \sum_n X(i+m, j+n) K(m,n)$$

where X is the input feature map, K is a learned convolutional kernel, and Y is the resulting output feature map at spatial position (i,j) ; such layers allow the network to learn localized cepstral-temporal patterns—

for example, abrupt-onset or repeated-onset structure associated with blocks and repetitions—without hand-specifying them. Batch normalization stabilizes and accelerates training by normalizing layer activations within each mini-batch [27]. Max pooling reduces the spatial resolution of feature maps, providing local translation invariance and reducing the parameter count of subsequent layers. Dropout regularizes the network by randomly deactivating a fraction of units during training, mitigating overfitting on a moderately sized dataset [28]. The rectified linear unit (ReLU), $f(x) = \max(0, x)$, is the standard nonlinearity used between convolutional/dense layers in this class of architecture. The final sigmoid activation,

$$\sigma(z) = 1 / (1 + e^{(-z)})$$

maps the network's final scalar output z to a probability in $[0, 1]$, which is thresholded at 0.5 to produce the binary Stuttered/Non-stuttered decision used throughout Section 5.

4. Experimental Setup

The experimental configuration below was recovered directly from the supplied code. Values that could not be determined from the supplied materials are marked [REQUIRED] rather than assumed.

Parameter	Value
Programming language	Python
Key libraries used	pandas, NumPy, librosa, soundfile, scikit-learn, TensorFlow/Keras, tqdm
Python / library versions	[REQUIRED]
Hardware (CPU/GPU)	[REQUIRED]
Random seed (data splitting)	42
Training epochs	[REQUIRED]
Training batch size	[REQUIRED]
Test-time inference batch size	64
Optimizer	[REQUIRED]
Learning rate	[REQUIRED]
Loss function	[REQUIRED] (binary cross-entropy is conventional for this architecture but unconfirmed)
Early stopping / LR scheduling	[REQUIRED]
Model checkpointing	Best model saved as <code>cnn_v2_best.keras</code> (confirmed)
Classification threshold	0.5 (confirmed in evaluation script)

Table 4b. Experimental configuration.

Algorithm 1 additionally names the Adam optimizer [29], a widely used adaptive stochastic gradient-based optimization method, for model training; however, as the table above indicates, this specific optimizer choice, along with the associated learning rate, could not be independently confirmed from the supplied training script and is accordingly treated as unconfirmed in this experimental-setup summary rather than asserted as verified. For reference, if binary cross-entropy loss were used, it is defined for a single example as

$$L = -[y \cdot \log(p) + (1-y) \cdot \log(1-p)]$$

where $y \in \{0, 1\}$ is the ground-truth binary label and p is the model's predicted probability that the recording is Stuttered. This equation is presented for completeness

given the sigmoid output layer and is not asserted as the confirmed training loss.

5. Results

5.1 Overall Performance

Precision, recall, and F1-score are reported alongside overall accuracy because, under class imbalance such as that present in this corpus, accuracy alone can obscure poor minority-class performance, and reporting complementary metrics is standard practice for imbalanced binary classification tasks [34]. Table 5 reports the proposed model's performance on the held-out, speaker-disjoint test set (2,152 recordings, 7 speakers, none of whom appear in the training or validation partitions).

Metric	Test Performance
Accuracy	91.54%
Precision (Stuttered)	71.77%
Recall (Stuttered)	96.35%
F1-score (Stuttered)	82.26%
ROC-AUC	96.89%

Table 5. Overall test-set performance.

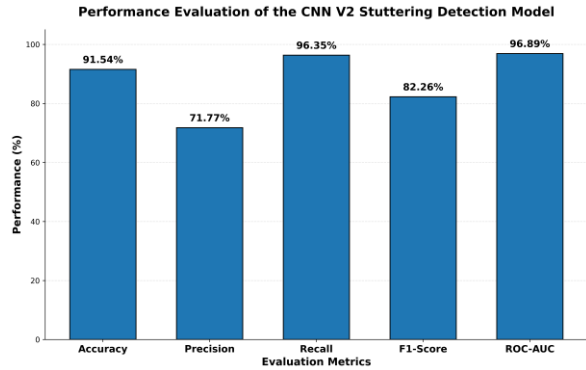


Fig. 10. Overall test-set performance of the CNN V2 stuttering-detection model: accuracy, precision, recall, F1-score, and ROC-AUC.

5.2 Class-Wise Performance

Class	Precision	Recall	F1-score	Support
Non-stuttered	98.98%	90.32%	94.45%	1,714
Stuttered	71.77%	96.35%	82.26%	438
Macro average	85.37%	93.33%	88.35%	2,152
Weighted average	93.44%	91.54%	91.97%	2,152

Table 6. Class-wise classification performance.

5.3 Confusion Matrix

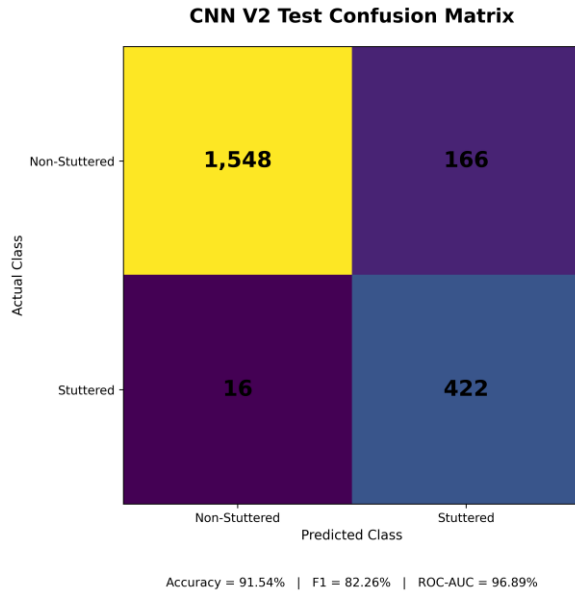


Fig. 11. CNN V2 test-set confusion matrix, with associated accuracy, F1-score, and ROC-AUC.

As numerical verification, using the standard definitions

$$\begin{aligned} \text{Accuracy} &= (TP+TN)/(TP+TN+FP+FN) & \text{Precision} &= TP/(TP+FP) \\ \text{Recall} &= TP/(TP+FN) & \text{F1} &= 2 \cdot P \cdot R / (P+R) \end{aligned}$$

and treating "Stuttered" as the positive class, the confusion matrix in Fig. 11 (TN = 1,548 Non-stuttered correctly classified; FP = 166 Non-stuttered misclassified as Stuttered; FN = 16 Stuttered misclassified as Non-stuttered; TP = 422 Stuttered correctly classified) reproduces the reported metrics: Accuracy = $(1,548+422)/2,152 = 91.54\%$; Precision = $422/(422+166) = 71.77\%$; Recall = $422/(422+16) = 96.35\%$; F1 = $2(0.7177)(0.9635)/(0.7177+0.9635) \approx 82.3\%$, matching the reported 82.26% within rounding. The per-class figures in Table 6 were verified against the same matrix in an identical manner (e.g., Non-stuttered precision = $1,548/(1,548+16) = 98.98\%$; Non-stuttered

recall = $1,548/1,714 = 90.32\%$) and were likewise found to be consistent. No numerical inconsistency was detected between the confusion matrix and the reported summary metrics.

5.4 ROC Analysis

The Receiver Operating Characteristic (ROC) curve characterizes the trade-off between the true positive rate (recall) and the false positive rate as the decision threshold on the sigmoid output is varied continuously, rather than fixed at 0.5 [35]. The Area Under the ROC Curve (ROC-AUC) summarizes this trade-off as a single scalar equal to the probability that the model assigns a higher Stuttered-class score to a randomly chosen Stuttered recording than to a randomly chosen Non-stuttered recording; a value of 1.0 indicates perfect separability and 0.5 indicates chance-level separability. The reported ROC-AUC of 96.89% indicates that the model's continuous output scores separate the two classes substantially better than the accuracy figure alone might suggest, and that performance at operating points other than the default 0.5 threshold—for instance, a threshold chosen to further increase recall for a screening application—remains available without retraining.

5.5 Training Behavior

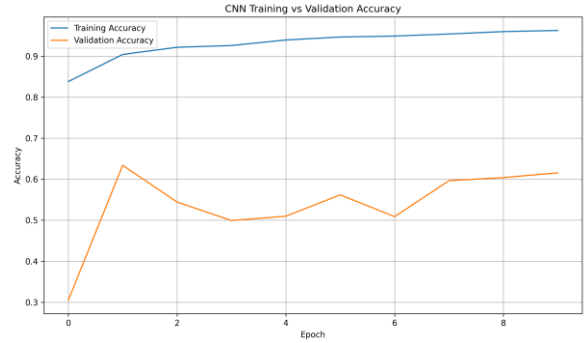


Fig. 12. Training vs. validation accuracy across training epochs for the CNN V2 model.

Validation accuracy behaves differently: it rises sharply from approximately 31% to approximately 63% between epoch 0 and epoch 1, but then remains largely within the 50%-62% range for the remainder of training rather than continuing to approach the training accuracy. Validation loss is correspondingly non-monotonic, falling to a minimum of approximately 0.92 at epoch 1, rising again through epochs 2-4, dipping briefly at epoch 5, spiking sharply to approximately 2.48 at epoch 6, and then declining to approximately 1.22 by epoch 9. Taken together, these curves show a persistent, non-closing gap between training and validation performance across

the plotted epochs, a pattern consistent with overfitting or optimization instability during training.

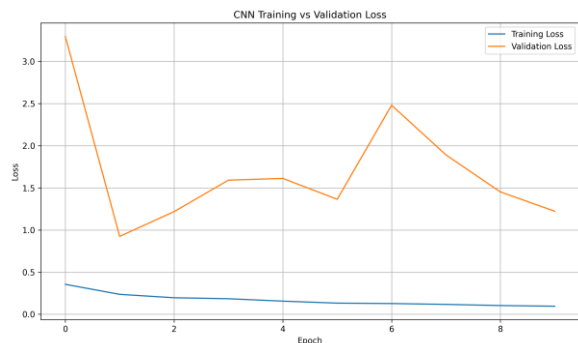


Fig. 13. Training vs. validation loss across training epochs for the CNN V2 model.

At face value, this pattern is difficult to reconcile with the substantially higher accuracy (91.54%) reported on the held-out test set in Section 5.1. Because the training/validation curves and the final test-set evaluation were supplied as separate artifacts for this paper, several explanations are possible—among them, that the plotted training run does not correspond exactly to the checkpoint evaluated in Section 5, or that the plotted validation split differs from the test split—but none of these could be independently confirmed from the supplied materials. Consistent with the reporting approach used throughout this paper, this discrepancy is therefore stated explicitly rather than silently reconciled, and is revisited as a limitation in Section 7.

6. Discussion

The proposed CNN achieves strong recall (96.35%) on the minority Stuttered class, meaning that of the 438 genuinely stuttered test recordings, only 16 were missed. For a screening or fluency-monitoring application, this property is desirable: a missed stuttering event (false negative) represents a case in which the system fails to flag speech that may warrant clinical attention, whereas a false positive (166 cases here, a Non-stuttered recording flagged as Stuttered) is comparatively lower-cost, since it can be reviewed and dismissed by a human evaluator without a missed detection. The precision of 71.77% reflects this asymmetry directly: the model is tuned, whether by design or as an emergent property of the class-imbalanced training distribution (1,443 Stuttered vs. 7,942 Non-stuttered recordings in training—see Table 3), toward higher sensitivity at some cost to specificity. The substantially higher ROC-AUC (96.89%) relative to the accuracy at the default threshold (91.54%) further indicates that the underlying probability estimates

separate the two classes well, and that the precision/recall balance could be adjusted post hoc via threshold selection if a different operating point were preferred for a particular deployment context.

The comparatively low false-negative count (16) alongside a higher false-positive count (166) is also consistent with the class imbalance in the training data noted above (approximately 5.5:1 Non-stuttered:Stuttered in training): with fewer Stuttered examples available, a boundary that favors recall over precision for the minority class is a plausible, if unconfirmed, outcome, since a false negative on so small a minority class carries disproportionate weight if the loss or evaluation criterion is sensitive to it, consistent with broader findings on how class imbalance shapes the behavior of CNN-based classifiers [37], [39]. Because the specific loss function and any class-weighting scheme used during training were not included in the supplied materials (Section 4), this explanation is offered as a plausible interpretation rather than a confirmed mechanism. The magnitude of imbalance observed here (approximately 5.5:1) is comparable to that reported for other stuttering corpora, where data-balancing choices have been shown to materially affect detection and classification outcomes [22]; reweighting strategies such as class-balanced or focal losses [30] and synthetic minority oversampling [36] have been proposed in the broader imbalanced-classification literature as potential mitigations, though none of these is confirmed to have been part of the training procedure used here.

The central methodological strength of this study is that all reported test-set numbers were obtained under a verified speaker-independent evaluation protocol (Section 3.5): the 7 test speakers do not appear in either the training or validation speaker sets, and this was checked programmatically rather than assumed. This distinguishes the reported 91.54% accuracy and 96.89% ROC-AUC from performance figures obtained under recording-level random splitting, where the same speaker's voice, recording device, and background acoustic conditions could appear in both training and test data and artificially inflate apparent performance. Because the underlying implementation explicitly verifies zero speaker overlap across partitions, the reported generalization is to acoustic patterns associated with fluency rather than to speaker identity per se, to the extent that the 7-speaker test set is representative of the broader population of speakers who do and do not stutter.

Relative to the deep-learning stuttering-detection literature summarized in Section 2, several methodological differences should be noted before any performance comparison is drawn. FluentNet [4] and StutterNet [5] address multi-class disfluency-type classification—for example, distinguishing prolongations from repetitions from blocks—on the UCLASS and/or LibriStutter corpora, a different and generally harder task than the binary Stuttered/Non-stuttered classification addressed here. The atrous-CNN

approach of Al-Banna et al. [11] likewise targets multi-class disfluency classification from log-Mel spectrogram input on UCLASS and FluencyBank. Because these prior systems differ in task formulation (binary vs. multi-class), dataset, and, in most cases, the granularity of speaker-independence verification, the numeric comparison in Table 7 is reported for context only and should not be interpreted as a controlled benchmark under identical conditions.

Study	Dataset	Task / Method	Accuracy	F1	AUC
Kourkounakis et al., FluentNet [4]	UCLASS, LibriStutter	Multi-class, CNN+BiLSTM+attn.	N/R*	N/R	N/R
Sheikh et al., StutterNet [5]	UCLASS (100+ spk.)	Multi-class, TDNN	N/R	N/R	N/R
Al-Banna et al. [11]	UCLASS, FluencyBank	Multi-class, 2D atrous CNN	N/R	N/R	N/R
Prabhu & Seliya [13]	SEP-28k	Binary/stutter ID, CNN	N/R	N/R	N/R
Proposed method	Project corpus (binary)	Binary, CNN on MFCC+ Δ + $\Delta\Delta$, spk.-indep.	91.54%	82.26%	96.89%

Table 7. Comparison with previously published stuttering-detection studies (not a controlled, identical-conditions comparison).

*N/R denotes that a directly comparable, single-number accuracy/F1/AUC figure was not confidently extractable from the cited sources without risking a mismatched task definition; per the instructions governing this paper, no values were estimated to fill these cells.

7. Limitations

No ablation study (e.g., removing delta/delta-delta features, varying MFCC count, or comparing against a log-Mel-spectrogram input) was performed as part of the materials supplied for this work, and none is claimed. Beyond this, the following limitations apply to the present study:

- **Dataset scale and speaker diversity.** The evaluation is based on only 44 unique speakers in total, with just 7 held out for testing and 7 for validation. Performance estimates on such a small speaker-level test set carry non-trivial variance.
- **Class imbalance.** The Stuttered class constitutes roughly 18% of the overall corpus and a similar proportion within each partition, which plausibly contributes to the precision/recall asymmetry discussed in Section 6, a challenge also documented in other stuttering-detection corpora [22].
- **Binary-label simplification.** Collapsing Fluent and Normal speech into a single Non-stuttered class discards the finer-grained multi-class disfluency-type information that several prior systems explicitly model [4], [5], [11].

- **Recording conditions and provenance.** The Normal-speech and stuttering-speech recordings originate from different source collections; residual channel or recording-condition differences cannot be ruled out as a confound.

- **Lack of external and cross-corpus validation.** All results are reported on a held-out partition of the same underlying corpus; no evaluation on an independent public benchmark (e.g., SEP-28k, UCLASS, FluencyBank) was performed.

- **Unreconciled training-dynamics evidence.** The supplied training/validation accuracy and loss curves (Fig. 12, Fig. 13) show validation accuracy plateauing at roughly 50%–62% while training accuracy approaches 96%, and a non-monotonic validation loss—a pattern indicative of overfitting or training instability that is not straightforwardly consistent with the 91.54% accuracy reported on the held-out test set in Section 5.1. Because the supplied materials do not document whether the plotted training run corresponds exactly to the checkpoint (cnn_v2_best.keras) evaluated in Section 5, this discrepancy is reported rather than resolved, and the relationship between the plotted training dynamics and the final test-set metrics should be treated with corresponding caution.

- **Incomplete architecture and hyperparameter disclosure.** The CNN’s internal layer configuration and training hyperparameters were not available in the supplied materials.

- **No clinical validation.** This system has not been evaluated against clinician-adjudicated ground truth or in a therapeutic setting, and no claim of clinical or diagnostic validity is made.

8. Future Work

Building on the present speaker-independent binary framework, several directions are identified as realistic extensions rather than as claims about the current system. The corpus could be expanded with additional speakers and recording conditions to reduce the variance associated with a 44-speaker evaluation and to improve robustness to channel variability. Cross-dataset evaluation—for example, training on the present corpus and testing on SEP-28k, UCLASS, or FluencyBank without fine-tuning—would provide a more stringent test of generalization than the current within-corpus, speaker-disjoint split. The binary task could be extended to multi-class disfluency-type classification (repetition, prolongation, block, interjection) to align more closely with clinically relevant categories, following the precedent set by FluentNet [4], StutterNet [5], and the atrous-CNN approach of Al-Banna et al. [11], later extended with an attention mechanism over heterogeneous acoustic features [17]. Architecturally, replacing or augmenting the MFCC-based CNN input with self-supervised speech representations (e.g., Wav2Vec2.0 embeddings, as explored for stuttering detection by Sheikh et al. [10], as well as by more recent self-supervised word-level stutter-detection models [23] and wav2vec 2.0-based dysfluency detection in stuttering-therapy speech [24]) or with transformer-based sequence models could be investigated as alternatives to the current CNN. Explainability techniques (e.g., saliency or attention visualization over the 120×313 feature tensor) could help clarify which cepstral-temporal regions drive the model's predictions. Finally, real-time inference feasibility and, at a substantially later stage and only in collaboration with qualified clinicians, prospective

clinical validation would be necessary before any diagnostic or therapeutic claims could be supported.

9. Conclusion

This paper presented a speaker-independent CNN-based framework for binary stuttering detection that classifies speech recordings as Non-stuttered or Stuttered. The three original recording categories present in the source data (Normal, Fluent, Dysfluent) were consolidated into this binary target using an explicit, code-verified transformation. Audio was standardized through a resampling, silence-trimming, duration-filtering, and amplitude-normalization pipeline, and represented for classification as a 120×313 tensor of 40 MFCCs together with their first- and second-order temporal derivatives—a representation explicitly distinguished in this paper from the waveform and log-Mel spectrogram visualizations used elsewhere in the project's figures for illustrative purposes. Speaker identities were recovered, grouped, and partitioned at the speaker level across 44 unique speakers so that no speaker appeared in more than one of the training, validation, and test sets, and this property was verified programmatically rather than assumed. On the resulting speaker-disjoint test set, the proposed CNN achieved 91.54% accuracy, 82.26% F1-score, and 96.89% ROC-AUC, with all reported metrics independently re-derived from the underlying confusion matrix and found to be internally consistent. The high recall obtained for the Stuttered class suggests the system is well suited to screening-oriented use cases in which missed stuttering events are more costly than false alarms, though this trade-off, the moderate scale of the speaker pool, and the absence of cross-corpus and clinical validation should be considered when interpreting these results. Future work extending this framework to multi-class disfluency typing, cross-corpus evaluation, and self-supervised acoustic representations is expected to further clarify the extent to which the reported generalization to unseen speakers extends to unseen recording conditions and populations.

References

- [1] B. Guitar, *Stuttering: An Integrated Approach to Its Nature and Treatment*. Philadelphia, PA, USA: Lippincott Williams & Wilkins, 2013.
- [2] A. Craig, E. Blumgart, and Y. Tran, "The impact of stuttering on the quality of life in adults who stutter," *J. Fluency Disord.*, vol. 34, no. 2, pp. 61–71, 2009, doi: 10.1016/j.jfludis.2009.05.002.
- [3] C. Lea, V. Mitra, A. Joshi, S. Kajarekar, and J. P. Bigham, "SEP-28k: A dataset for stuttering event detection from podcasts with people who stutter," in *Proc. IEEE Int. Conf. Acoust., Speech, Signal Process. (ICASSP)*, Toronto, ON, Canada, 2021, pp. 6798–6802, doi: 10.1109/ICASSP39728.2021.9413520.
- [4] T. Kourkounakis, A. Hajavi, and A. Etemad, "FluentNet: End-to-end detection of stuttered speech disfluencies with deep learning," *IEEE/ACM Trans. Audio, Speech, Lang. Process.*, vol. 29, pp. 2986–2999, 2021, doi: 10.1109/TASLP.2021.3110146.
- [5] S. A. Sheikh, Md. Sahidullah, F. Hirsch, and S. Ouni, "StutterNet: Stuttering detection using time delay neural network," in *Proc. 29th Eur. Signal Process. Conf. (EUSIPCO)*, Dublin, Ireland, 2021, pp. 426–430, doi: 10.23919/EUSIPCO54536.2021.9616063.
- [6] P. Howell, S. Davis, and J. Bartrip, "The UCLASS archive of stuttered speech," *J. Speech, Lang., Hear. Res.*, vol. 52, no. 2, pp. 556–569, 2009, doi: 10.1044/1092-4388(2009/07-0129).
- [7] R. Alnashwan, N. Alhakbani, A. Al-Nafjan, A. Almudhi, and W. Al-Nuwaier, "Computational intelligence-based stuttering detection: A systematic review," *Diagnostics*, vol. 13, no. 23, art. 3537, 2023, doi: 10.3390/diagnostics13233537.
- [8] S. A. Sheikh, M. Sahidullah, F. Hirsch, and S. Ouni, "Machine learning for stuttering identification: Review, challenges and future directions," *Neurocomputing*, vol. 514, pp. 385–402, 2022, doi: 10.1016/j.neucom.2022.10.015.
- [9] S. A. Sheikh, M. Sahidullah, F. Hirsch, and S. Ouni, "Advancing stuttering detection via data augmentation, class-balanced loss and multi-contextual deep learning," *IEEE J. Biomed. Health Inform.*, vol. 27, no. 5, pp. 2553–2564, 2023, doi: 10.1109/JBHI.2023.3248281.
- [10] S. A. Sheikh, M. Sahidullah, F. Hirsch, and S. Ouni, "Introducing ECAPA-TDNN and Wav2Vec2.0 embeddings to stuttering detection," *arXiv preprint arXiv:2204.01564*, 2022.
- [11] A.-K. Al-Banna, E. Edirisinghe, and H. Fang, "Stuttering detection using atrous convolutional neural networks," in *Proc. 13th Int. Conf. Inf. Commun. Syst. (ICICS)*, Irbid, Jordan, 2022, pp. 252–256.
- [12] M. Jouaiti and K. Dautenhahn, "Dysfluency classification in stuttered speech using deep learning for real-time applications," in *Proc. IEEE Int. Conf. Acoust., Speech, Signal Process. (ICASSP)*, Singapore, 2022, pp. 6482–6486.
- [13] Y. Prabhu and N. Seliya, "A CNN-based automated stuttering identification system," in *Proc. 21st IEEE Int. Conf. Mach. Learn. Appl. (ICMLA)*, Nassau, Bahamas, 2022, pp. 1601–1605.
- [14] L. Barrett, J. Hu, and P. Howell, "Systematic review of machine learning approaches for detecting developmental stuttering," *IEEE/ACM Trans. Audio, Speech, Lang. Process.*, vol. 30, pp. 1160–1172, 2022, doi: 10.1109/TASLP.2022.3155295.
- [15] S. A. Sheikh, Md. Sahidullah, F. Hirsch, and S. Ouni, "Robust stuttering detection via multi-task and adversarial learning," in *Proc. 30th Eur. Signal Process. Conf. (EUSIPCO)*, 2022, pp. 190–194.
- [16] M. Abubakar, M. Mujahid, K. Kanwal, S. Iqbal, M. N. Asghar, and A. Alaulamie, "StutterNet: Stuttering disfluencies detection in synthetic speech signals via mel frequency cepstral coefficients features using deep learning," *IEEE Access*, vol. 12, pp. 99308–99320, 2024, doi: 10.1109/ACCESS.2024.3429343.
- [17] A.-K. Al-Banna, H. Fang, and E. Edirisinghe, "A novel attention model across heterogeneous features for stuttering event detection," *Expert Syst. Appl.*, vol. 244, art. 122967, 2024, doi: 10.1016/j.eswa.2023.122967.
- [18] N. Alhakbani, R. Alnashwan, A. Al-Nafjan, and A. Almudhi, "Automated stuttering detection using deep learning techniques," *J. Clin. Med.*, vol. 14, no. 10, art. 3552, 2025, doi: 10.3390/jcm14103552.
- [19] A.-K. Al-Banna, E. Edirisinghe, H. Fang, and W. Hadi, "Stuttering disfluency detection using machine learning approaches," *J. Inf. Knowl. Manage.*, vol. 21, no. 2, 2022, doi: 10.1142/S0219649222500204.
- [20] T. Kourkounakis, A. Hajavi, and A. Etemad, "Detecting multiple speech disfluencies using a deep residual network with bidirectional long short-term memory," in *Proc. IEEE Int. Conf. Acoust., Speech, Signal Process. (ICASSP)*, 2020, pp. 6089–6093, doi: 10.1109/ICASSP40776.2020.9053893.
- [21] S. Alharbi, M. Hasan, A. J. H. Simons, S. Brumfitt, and P. Green, "Sequence labeling to detect stuttering events in read speech," *Comput. Speech Lang.*, vol. 62, art. 101052, 2020, doi: 10.1016/j.csl.2019.101052.
- [22] A. Batra, P. Shrivastava, and P. K. Das, "Does data balancing impact stutter detection and classification?," in *Distributed Computing and Intelligent Technology (ICDCIT 2025)*, Lecture Notes in Computer Science, vol. 15507, Springer, Cham, 2025, doi: 10.1007/978-3-031-81404-4_16.
- [23] Y.-J. Shih, Z. Gkalitsiou, A. G. Dimakis, and D. Harwath, "Self-supervised speech models for word-level stuttered speech detection," in *Proc. IEEE Spoken Language Technology Workshop (SLT)*, 2024, pp. 937–944, doi: 10.1109/SLT61566.2024.10832220.

- [24] S. P. Bayerl, A. W. von Gudenberg, F. Hönig, E. Nöth, and K. Riedhammer, "Detecting dysfluencies in stuttering therapy using wav2vec 2.0," in *Proc. Interspeech 2022*, pp. 2228–2232, doi: 10.21437/Interspeech.2022-630.
- [25] S. Davis and P. Mermelstein, "Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences," *IEEE Trans. Acoust., Speech, Signal Process.*, vol. 28, no. 4, pp. 357–366, 1980, doi: 10.1109/TASSP.1980.1163420.
- [26] S. Furui, "Speaker-independent isolated word recognition using dynamic features of speech spectrum," *IEEE Trans. Acoust., Speech, Signal Process.*, vol. 34, no. 1, pp. 52–59, 1986, doi: 10.1109/TASSP.1986.1164788.
- [27] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proc. 32nd Int. Conf. Mach. Learn. (ICML)*, 2015, pp. 448–456.
- [28] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *J. Mach. Learn. Res.*, vol. 15, no. 1, pp. 1929–1958, 2014.
- [29] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. 3rd Int. Conf. Learn. Represent. (ICLR)*, 2015.
- [30] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, 2017, pp. 2980–2988, doi: 10.1109/ICCV.2017.324.
- [31] A. B. Nassif, I. Shahin, I. Attili, M. Azzeh, and K. Shaalan, "Speech recognition using deep neural networks: A systematic review," *IEEE Access*, vol. 7, pp. 19143–19165, 2019, doi: 10.1109/ACCESS.2019.2896880.
- [32] S. P. Bayerl, D. Wagner, E. Nöth, T. Bocklet, and K. Riedhammer, "The influence of dataset partitioning on dysfluency detection systems," in *Text, Speech, and Dialogue (TSD 2022)*, *Lecture Notes in Computer Science*, Springer, Cham, 2022.
- [33] V. Zayats, M. Ostendorf, and H. Hajishirzi, "Disfluency detection using a bidirectional LSTM," in *Proc. Interspeech 2016*; arXiv:1604.03209.
- [34] J. Davis and M. Goadrich, "The relationship between precision-recall and ROC curves," in *Proc. 23rd Int. Conf. Mach. Learn. (ICML)*, 2006, pp. 233–240, doi: 10.1145/1143844.1143874.
- [35] T. Fawcett, "An introduction to ROC analysis," *Pattern Recognit. Lett.*, vol. 27, no. 8, pp. 861–874, 2006, doi: 10.1016/j.patrec.2005.10.010.
- [36] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *J. Artif. Intell. Res.*, vol. 16, pp. 321–357, 2002, doi: 10.1613/jair.953.
- [37] M. Buda, A. Maki, and M. A. Mazurowski, "A systematic study of the class imbalance problem in convolutional neural networks," *Neural Netw.*, vol. 106, pp. 249–259, 2018, doi: 10.1016/j.neunet.2018.07.011.
- [38] H. Purwins, B. Li, T. Virtanen, J. Schlüter, S.-Y. Chang, and T. Sainath, "Deep learning for audio signal processing," *IEEE J. Sel. Topics Signal Process.*, vol. 13, no. 2, pp. 206–219, 2019, doi: 10.1109/JSTSP.2019.2908700.
- [39] H. He and E. A. Garcia, "Learning from imbalanced data," *IEEE Trans. Knowl. Data Eng.*, vol. 21, no. 9, pp. 1263–1284, 2009, doi: 10.1109/TKDE.2008.239.